

F10: Datastrukturer

Malmö universitet, Institutionen för datavetenskap och medieteknik

Innehåll

- Repetition
- Stackar
- Köer
- Länkade listor
- Sammanfattning



Repetition

Repetition

- Klasser kan referera till objekt av samma klass
- Ett oinitierat objekt är *null*
- Java har en automagisk *garbage collection*
- Arrayer är en sammanhängande del i minnet
 - Detta kan göra det jobbigt att expandera arrayen



Stackar

Ett vardagsexempel på en stack

- I köksskåpet har de flesta av oss en hög med tallrikar
- När du lägger till nya tallrikar läggs dessa överst (som regel)
- När du tar en tallrik plockar du ut den översta tallriken
- Den senast inlagda tallriken är alltså den första att plockas ut



Stackar i kod-sammanhang

- Callstacken
 - Håller reda på i vilken ordning metoder har kallats
- Undo-funktioner
 - Håller reda på i vilken ordning handlingar ha skett
- Sidhistoriken i en webbläsare/filhanterare
 - När du trycker bakåt

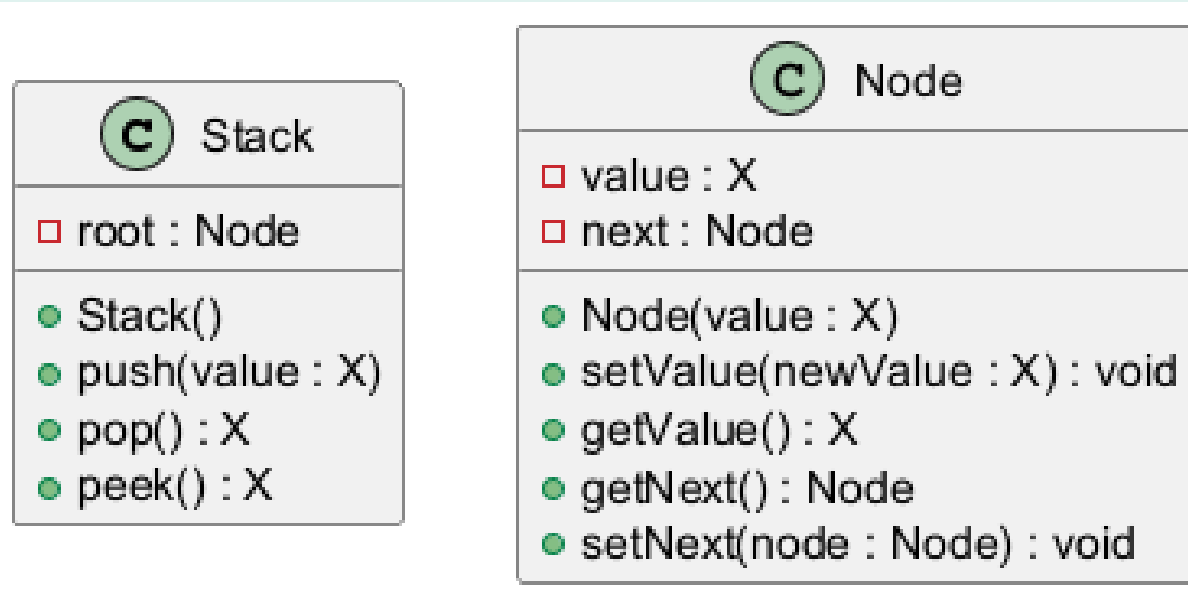
Last In First Out

- Principen Last In First Out innebär att den som varit i stacken kortast tid är den som ska försvinna först
- Det är denna princip som används på svenska arbetsplatser när det gäller avskedningar

Noden

- Värdena i en stack sparas i noder
- Noden vet två saker:
 - Sitt egna värde
 - Vilken nod som kommer efteråt

Klassdiagram



Attribut i en stack

- En stack har som regel ett attribut
 - Start-noden (ofta kallad *root*)
- En stack kan också ha ett attribut som håller reda på antalet element

Metoder i en stack

- `push(value)`
 - Lägger till ett element överst/först i stacken
- `pop() : value`
 - Plockar bort och returnerar det översta/första elementet
- `peek() : value`
 - Tittar på det första värdet i stacken, utan att ta bort det

En stack i kod (konstruktor)

```
public class Stack {  
    private Node root;  
  
    public Stack(){  
        root = null;  
    }  
}
```

En stack i kod (push)

```
public void push(int value){  
    Node temp = root;  
    root = new Node(value);  
    root.setNext(temp);  
}
```


En stack i kod (pop)

```
public int pop(){  
    int value = root.getValue();  
    root = root.getNext();  
    return value;  
}
```

En stack i kod (peek)

```
public int peek(){  
    return root.getValue();  
}
```

Det stora kluriga

- Det finns framförallt två klurigheter med stackar
 - Ta bort element utan att ta bort allt
 - Att förstå hur stacken hänger ihop
- Lösningen till att förstå är att göra
- En annan sak som kan vara klurig är att hantera specialfallet när stacken är tom



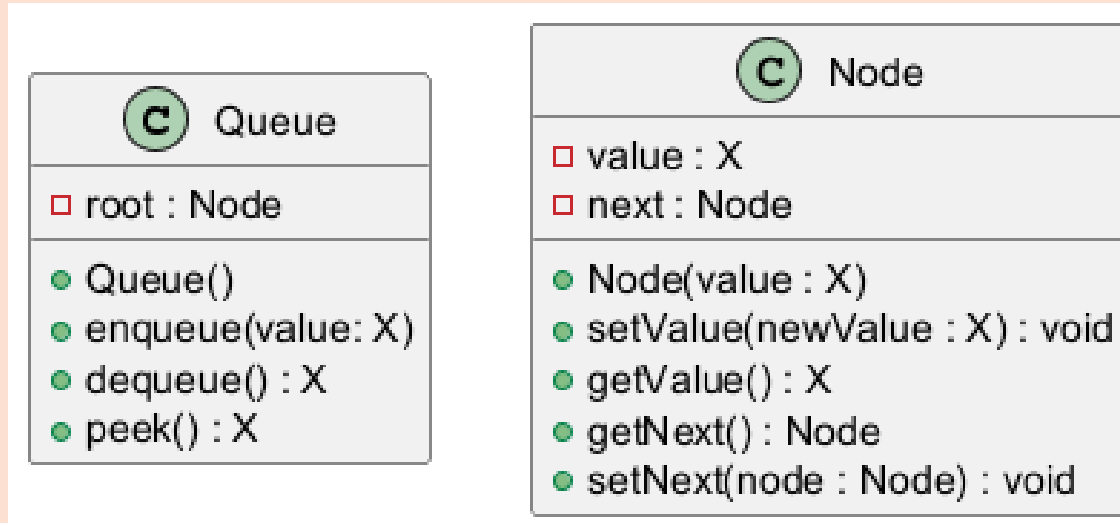
Köer

Köer in real life

- När du kommer fram till kassan i en butik ställer du dig sist i kön
- Kassören behandlar den som stått i kön längst



Klassdiagram



Attribut i en kö

- Köer och stackar har samma attribut
 - Start-element (ofta kallat *root*)
 - Antal element

Metoder i en kö

- `enqueue(value)`
- Lägger till ett element sist i kön
- `dequeue() : value`
- Tar bort och returnerar det första elementet i kön
- `peek() : value`
- Tittar på det första elementet i kön utan att ta bort det

En kö i kod-sammanhang

- Utskriftskö
 - Dokument ska komma i kronologisk ordning
- Allt annat där kronologi är viktigt

En kö i kod (konstruktor)

```
public class Queue {  
    private Node root;  
  
    public Queue(){  
  
    }  
}
```

En kö i kod (enqueue)

```
public void enqueue(int value){  
    if (this.root == null){  
        this.root = new Node(value);  
    }  
    else {  
        Node current = this.root;  
        while (current.getNext() != null) {  
            current = current.getNext();  
        }  
        current.setNext(new Node(value));  
    }  
}
```

En kö i kod (dequeue)

```
public int dequeue(){  
    int out = this.root.getValue();  
    this.root = this.root.getNext();  
    return out;  
}
```

Klurigheter med köer

- Klurigheterna är ofta de samma som med stackar
 - Hur i allsindrar fungerar något?
 - Hur plockar jag bort det första elementet utan att tappa bort resten?
 - Hur når jag det sista elementet för att lägga till?
 - Hur hanterar jag en tom kö?



Länkade listor

Länkade listor

- En länkad lista påminner om en array
- Du kan lägga till element på valfri plats i listan
- Du kan ta bort element på valfri plats
- Du kan titta på element på valfri plats

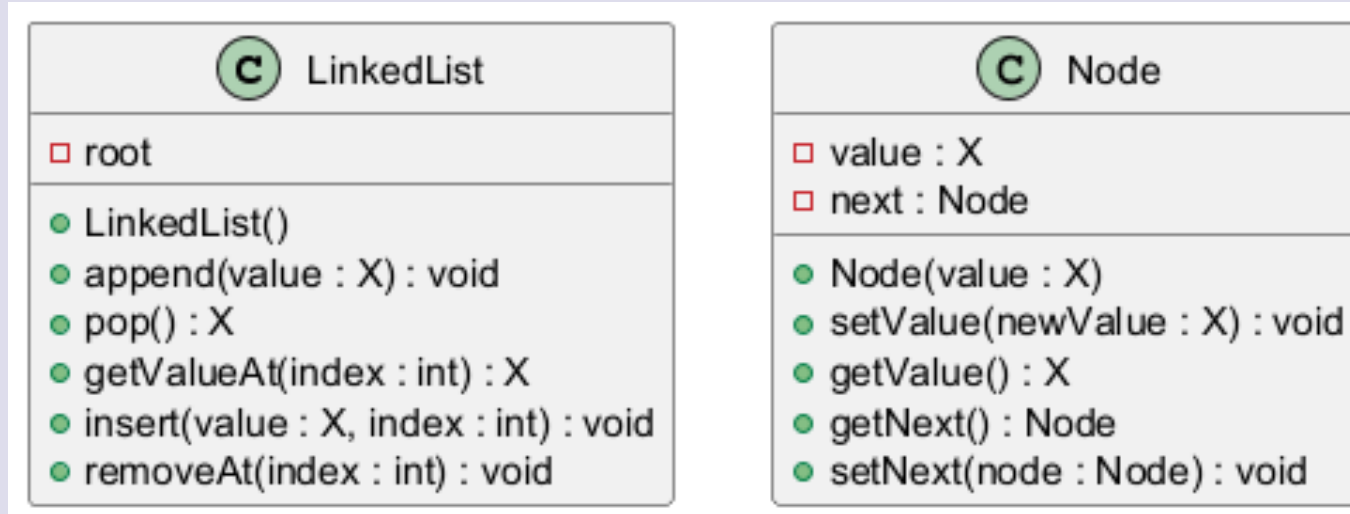
Minne — arrayer

- I Java har arrayer en bestämd storlek vid initialisering
- Detta innebär att datorn vet hur mycket minne arrayen behöver och allokerar detta.
- Denna allokering är en sammanhängande del i minnet
- Behöver vi utöka arrayen behöver vi allokera en ny sammanhängande del i minnet
- Detta gör att all data i arrayen behöver flyttas, och detta tar tid
- Alternativt behöver vi ha en stor array med tomma platser

Minne — länkade listor

- En länkad lista är *inte* sparad som en sammanhängande del i minnet
- Detta gör det lätt att ändra storleken på listan
- En länkad lista innehåller inga tomma platser (det går att implementera med tomma platser)
- Varje nytt element får sin egna plats i minnet
 - Detta gäller även köer och stackar

Klassdiagram



Attribut i en länkad lista

- Länkade listor har samma attribut som köer och stackar
 - En root nod
 - Eventuellt en int för storlek (för listor är detta ett relevant attribut, det gör framför allt pop() lättare)

Metoder i en länkad lista

- `append(value)`
 - Lägger till ett element i slutet av listan
- `pop() : value`
 - Tar bort och returnerar det sista värdet i listan
- `getValueAt(index : int) : value`
 - Returnerar värdet på plats `index`
- `insert(value, index : int)`
 - Stoppar in värdet `value` på platsen `index`
- `removeAt(index : int)`
 - Tar bort värdet på plats `index`

Länkade listor i kod (konstruktor)

```
public class LinkedList {  
    private Node root;  
  
    public LinkedList(){  
  
    }  
}
```

Länkade listor i kod (append)

```
public void append(int value){  
    if (this.root == null){  
        this.root = new Node(value);  
    }  
    else{  
        Node current = this.root;  
        while (current.getNext() != null){  
            current = current.getNext();  
        }  
        current.setNext(new Node(value));  
    }  
}
```

Länkade listor i kod (pop)

```
public int pop() {  
    int out = 0;  
    if (this.root.getNext() == null) {  
        out = this.root.getValue();  
        this.root = null;  
    }  
    else {  
        Node current = this.root;  
        while (current.getNext().getNext() != null) {  
            current = current.getNext();  
        }  
        out = current.getNext().getValue();  
        current.setNext(null);  
    }  
    return out;  
}
```

Länkade listor i kod (getValueAt)

```
public int getValueAt(int index){  
    int i = 0;  
    Node current = this.root;  
    while (i < index){  
        current = current.getNext();  
        i++;  
    }  
    return current.getValue();  
}
```


Länkade listor i kod (insert)

```
public void insert(int value, int index){
    int i = 0;
    Node newNode = new Node(value);
    if (index == 0){
        newNode.setNext(this.root);
        this.root = newNode;
    }
    else {
        Node current = this.root;
        while (i < index - 1) {
            current = current.getNext();
            i++;
        }
        newNode.setNext(current.getNext());
        current.setNext(newNode);
    }
}
```

Länkade listor i kod (removeAt)

```
public void removeAt(int index){
    int i = 0;
    if (index == 0){
        this.root = root.getNext();
    }
    else{
        Node current = this.root;
        while (i < index-1){
            current = current.getNext();
            i++;
        }
        current.setNext(current.getNext().getNext());
    }
}
```

Klurigheter med länkade listor

- Klurigheterna är ofta de samma som med stackar och köer
 - Hur i allsindrar fungerar något?
 - Hur plockar jag bort det första elementet utan att tappa bort resten?
 - Hur tar jag bort något i mitten utan att tappa bort resten?
 - Hur lägger jag till något i mitten utan att tappa bort resten?
 - Hur hanterar jag en tom lista?



Sammanfattning

Sammanfattning

- Stackar är Last In First Out (LIFO)
- Köer är First In First Out (FIFO)
- Listor är som arrayer fast med dynamisk längd

Noder

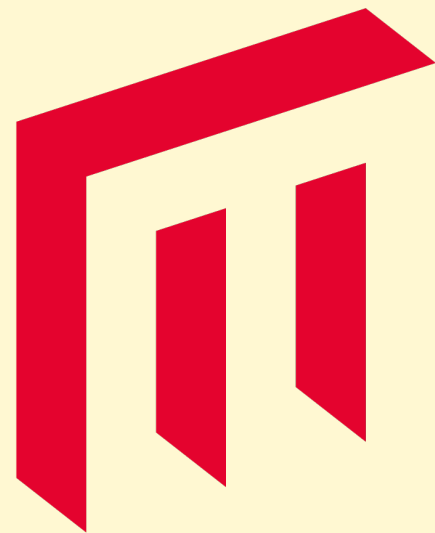
- Alla tre strukturerna bygger på att det finns noder
- Noderna håller reda på sitt värde och nästa nod
- En nod kan också hålla reda på föregående nod (exempelvis en dubbellänkad lista, eller ett träd)

Minneshantering

- Arrayer sparas i en sammanhängande del av datorminnet
 - Detta kan göra det svårt att utöka arrayen
- I Stackar, Köer, och Listor sparas varje element på sin egna plats
 - Detta gör det lättare att utöka strukturerna

Rekommenderad läsning

- Deitel & Deitel, kapitel 21, 856–909
- Aldous Huxley, *Brave New World*



**MALMÖ
UNIVERSITET**

